

RAG AGAINST THE MACHINE — 42 PROJECT SERIES

VOLUME 2 OF 3

Implementation Guide

The complete build, file by file: bootstrap, models, chunkers, index, CLI, generation — every line typed, linted, and explained

Covers: uv + pyproject + Makefile bootstrap • pydantic data models • corpus discovery • the analyzer
• Markdown and Python AST chunkers • BM25 index build and persistence • the retriever • the Fire
CLI • crash-free edge handling • Qwen3-0.6B answer generation • self-evaluation • tests and
troubleshooting

Companion study guide — subject version 2.0 — not an official 42 document

Contents

Project Bootstrap	4
Repository layout	4
pyproject.toml and uv	4
The Makefile	5
src/__init__.py and src/__main__.py	6
Data Models with pydantic	8
Three details that will bite you if skipped	9
Loading and saving with validation at the boundary	9
Walking the Corpus	11
Why each decision	12
The Analyzer	13
Reading the subtleties	14
The Markdown / Text Chunker	15
The algorithm	15
Design notes	17
The Python AST Chunker	18
Why it is written this way	20
Building and Persisting the BM25 Index	21
Choices you should be able to defend	22
The Retriever	24
Line-by-line commentary	25
Batch search over a dataset	25
The CLI with Python Fire	26
Fire behaviours worth knowing before the defense	27
Robustness: Never Crash	29
Walking the subject's edge-case list	29
Answer Generation with Qwen3-0.6B	31
Operational notes	33
Your Own evaluate, and the Full Pipeline Run	35
The four-command run, exactly as the defense will execute it	36
Anatomy of the Datasets	38
Reading them programmatically	38

The question style	39
A hand-written results file demystifies the grader	39
How a Query Flows: A Guided Code Reading	40
uv run python -m src index --max_chunk_size 2000	40
uv run python -m src search_dataset --dataset_path ... --k 10 --save_directory	40
Where each failure would surface	41
Passing the Gates: flake8, mypy, Docstrings	42
What each mypy flag actually enforces	42
The five patterns that keep this codebase green	42
A worked mypy failure and its fix	43
Testing What Matters	44
1 — Chunker invariants (the highest-value tests you will write)	44
2 — Analyzer contract	45
3 — Retrieval sanity and the metric	45
Troubleshooting the Build	47
Build Checklist and Bridge to Volume 3	49
The mandatory-part checklist	49
What Volume 3 adds	49
Submission hygiene	49

CHAPTER 1

Project Bootstrap

The reviewer and the moulinette only run `uv sync`. Everything in this chapter exists so that this one command is enough.

This volume builds the complete system, in dependency order, with every file shown in full and every line justified. The code is written to pass **flake8** and the subject's exact **mypy** invocation, to use **pydantic** for data models, **Fire** for the CLI, and **tqdm** for progress — the four hard tool requirements. Read with Volume 1 open: every design decision here was argued there, and cross-references say where.

Repository layout

The subject prescribes the evaluation layout exactly; deviating breaks the reference exam scripts and “the corresponding checks are considered failed by design”. The target tree:

Project tree — the evaluator's scripts assume exactly this

```
Rag-42/
+-- Makefile
+-- README.md
+-- pyproject.toml
+-- uv.lock
+-- .python-version      # e.g. 3.12
+-- .gitignore
+-- src/
|   +-- __init__.py
|   +-- __main__.py      # python -m src -> Fire CLI
|   +-- models.py        # pydantic data models
|   +-- analyzer.py      # text -> terms (one function, two call sites)
|   +-- chunkers.py      # markdown/text + python AST chunkers
|   +-- indexer.py       # corpus walk, chunking, BM25 stats, persist
|   +-- retriever.py     # BM25 scoring, top-k, batch search
|   +-- generator.py     # Qwen3-0.6B answer generation
|   `-- cli.py           # the Fire command class
+-- data/
|   +-- raw/vllm-0.10.1/  # provided corpus (never committed)
|   +-- processed/       # your index (generated)
|   +-- datasets/{Unanswered,Answered}Questions/
|   `-- output/...       # generated results
+-- tests/               # pytest (not graded, still write them)
```

Watch out — Do not commit `data/raw/`, `data/processed/`, `data/output/`, or model weights: the subject forbids large files in the repository, and the evaluator regenerates them. Your `.gitignore` must cover them from the first commit.

pyproject.toml and uv

`uv` resolves and installs dependencies from `pyproject.toml` into a local `.venv` and pins exact versions in `uv.lock`. Both files must live at the repository root and both must be committed — `uv sync`

reproduces the environment from them. A complete, sufficient configuration:

pyproject.toml — minimal and complete

```
[project]
name = "rag42"
version = "1.0.0"
description = "RAG system over the vLLM codebase (42 project)"
requires-python = ">=3.10"
dependencies = [
    "pydantic>=2.7",
    "fire>=0.6",
    "tqdm>=4.66",
    "transformers>=4.51",      # Qwen3 support begins at 4.51
    "torch>=2.3",
    "accelerate>=0.30",
]

[project.optional-dependencies]
dev = ["flake8>=7", "mypy>=1.10", "pytest>=8"]

[tool.mypy]
python_version = "3.10"

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Workflow commands you will use daily: **uv sync** (create/refresh the environment exactly from the lock file; add `--extra dev` for the linters), **uv add pkg** (add a dependency and update both files), **uv run python -m src ...** (run inside the environment without manual activation). Commit `uv.lock` every time it changes; an out-of-date lock is the classic “works on my machine, fails at defense” failure.

Note — Why `transformers >= 4.51`: Qwen3's architecture (`qwen3`) was added in that release; older versions raise `KeyError: 'qwen3'` when loading the model — a mysterious-looking crash with a one-line fix. Pin it now.

The Makefile

Five mandatory rules (install, run, debug, clean, lint) plus the recommended `lint-strict`. The lint flags are copied verbatim from the subject — they are part of the contract:

Makefile — rules named exactly as the subject demands (recipe lines are indented with REAL TAB characters, shown here as spaces)

```
.PHONY: install run debug clean lint lint-strict

install:
    uv sync --extra dev

run:
    uv run python -m src

debug:
    uv run python -m pdb -m src

clean:
    find . -type d -name __pycache__ -exec rm -rf {} +
    rm -rf .mypy_cache .pytest_cache .ruff_cache

lint:
    uv run flake8 .
    uv run mypy . --warn-return-any --warn-unused-ignores \
        --ignore-missing-imports --disallow-untyped-defs \
        --check-untyped-defs

lint-strict:
    uv run flake8 .
    uv run mypy . --strict
```

Add a `.flake8` file excluding the corpus and virtual environment, or `flake8` will lint all of vLLM's source (tens of thousands of findings that are not yours):

.flake8 — scope the linter to your code

```
[flake8]
max-line-length = 99
exclude = .venv,data,build,dist,__pycache__
extend-ignore = E203,W503
```

Mirror the same exclusions for `mypy` in `pyproject.toml` (`[tool.mypy] exclude = "(.venv|data)/"`) so make lint runs clean out of the box. Run `make lint` after every chapter of this volume — retrofitting types onto a finished project is misery; keeping a clean bill after each file is trivial.

`src/__init__.py` and `src/__main__.py`

The package marker can be empty (a docstring keeps linters happy). `__main__.py` is the `uv run python -m src` entry point; it hands the CLI class to Fire (built in chapter 9) and is deliberately tiny so the interesting logic stays testable:

```
src/__init__.py
"""RAG against the machine - src package."""
```

src/__main__.py — complete

```
"""Entry point: uv run python -m src <command> [options]."""
import fire

from src.cli import RagCli

def main() -> None:
    """Dispatch the command line to the Fire CLI."""
    fire.Fire(RagCli)

if __name__ == "__main__":
    main()
```

CHAPTER 2

Data Models with pydantic

The subject gives you the schemas; pydantic turns them into validated, typed, JSON-round-trippable classes.

pydantic models validate on construction: feed `answer=None` into a model whose field is `answer: str` and you get an immediate, precise `ValidationError` instead of a corrupt JSON file discovered by the `moulinette`. They also serialize (`model_dump_json`) and parse (`model_validate_json`) for free, which is your whole I/O layer. The complete models file, matching the subject's definitions exactly and adding nothing they forbid:

src/models.py — complete, subject-conformant

```
"""Pydantic data models exchanged between pipeline stages."""
import uuid
from typing import List, Union

from pydantic import BaseModel, Field

class MinimalSource(BaseModel):
    """A source location: file path plus character span."""

    file_path: str
    first_character_index: int
    last_character_index: int

class UnansweredQuestion(BaseModel):
    """A question without ground-truth answer or sources."""

    question_id: str = Field(
        default_factory=lambda: str(uuid.uuid4())
    )
    question: str

class AnsweredQuestion(UnansweredQuestion):
    """A question with its reference answer and sources."""

    sources: List[MinimalSource]
    answer: str

class RagDataset(BaseModel):
    """A dataset of answered and/or unanswered questions."""

    rag_questions: List[Union[AnsweredQuestion, UnansweredQuestion]]

class MinimalSearchResults(BaseModel):
    """Retrieved sources for one question."""

    question_id: str
```

```

question: str
retrieved_sources: List[MinimalSource]

class MinimalAnswer(MinimalSearchResults):
    """Retrieved sources plus the generated answer."""

    answer: str

class StudentSearchResults(BaseModel):
    """Output of search_dataset: all questions' results."""

    search_results: List[MinimalSearchResults]
    k: int

class StudentSearchResultsAndAnswer(BaseModel):
    """Output of answer_dataset: results plus answers."""

    search_results: List[MinimalAnswer]
    k: int

```

Three details that will bite you if skipped

1. **Union order matters.** In `RagDataset` the union is written `AnsweredQuestion | UnansweredQuestion` with the *subclass first*. pydantic tries members left to right (smart mode helps, but be explicit): with `UnansweredQuestion` first, an answered question would happily parse as unanswered and silently drop its sources and answer — and your `evaluate` command would divide by zero ground-truth sources.
2. **question_id is a string.** The default factory produces `str(uuid.uuid4())`; dataset files carry arbitrary id strings. Never parse it as a UUID type — compare as opaque strings, exactly like file paths.
3. **Models are a foundation, not a cage.** The subject explicitly allows extra fields and models (e.g. adding a score to your internal results). If you extend, extend by subclassing internal-only models; keep the *serialized* output limited to what the *moulinette* validates, and it will accept a superset anyway — but the safest posture is: internal riches, minimal wire format.

Loading and saving with validation at the boundary

All file I/O goes through two tiny helpers so that every byte entering or leaving the system passes model validation — put them in `models.py` or a small `io.py`:

Validated I/O helpers — the only place JSON touches disk

```
from pathlib import Path
from typing import Type, TypeVar

M = TypeVar("M", bound=BaseModel)

def load_model(path: Path, model: Type[M]) -> M:
    """Read and validate a JSON file as the given model."""
    with path.open("r", encoding="utf-8") as fh:
        return model.model_validate_json(fh.read())

def save_model(path: Path, instance: BaseModel) -> None:
    """Serialize a model to pretty JSON, creating parents."""
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as fh:
        fh.write(instance.model_dump_json(indent=2))
```

Note — pydantic v2 API names used throughout: `model_validate_json` / `model_dump_json` (v1's `parse_raw/json()` are deprecated). If you find yourself writing `.dict()` or `.json()`, you are reading v1 tutorials — check the version, the subject's ecosystem is v2.

CHAPTER 3

Walking the Corpus

Deciding what to index is a recall decision disguised as a file-system loop.

Volume 1 chapter 13 argued the policy: index Python and Markdown fully, carry small text-like files too, skip binaries and monsters. This chapter implements it. The discovery function is boring by design — every branch is a policy from Volume 1 with a measurable reason:

src/indexer.py (part 1) — discovery and reading

```

"""Corpus discovery: which files get indexed, and how to read them."""
from pathlib import Path
from typing import Iterator, Tuple

RAW_ROOT = Path("data/raw")

PYTHON_EXT = {".py"}
MARKDOWN_EXT = {".md"}
TEXT_EXT = {
    ".txt", ".rst", ".yaml", ".yml", ".toml", ".cfg",
    ".ini", ".sh", ".json", ".jinja",
}
SKIP_DIRS = {".git", "__pycache__", ".github", "node_modules"}
MAX_FILE_BYTES = 2_000_000 # 2 MB: nothing useful is bigger

def iter_corpus_files(root: Path = RAW_ROOT) -> Iterator[Tuple[Path, str]]:
    """Yield (path, kind) for every indexable file under root.

    kind is one of 'python', 'markdown', 'text'.
    """
    for path in sorted(root.rglob(".*")):
        if not path.is_file():
            continue
        if any(part in SKIP_DIRS for part in path.parts):
            continue
        try:
            if path.stat().st_size > MAX_FILE_BYTES:
                continue
        except OSError:
            continue
        suffix = path.suffix.lower()
        if suffix in PYTHON_EXT:
            yield path, "python"
        elif suffix in MARKDOWN_EXT:
            yield path, "markdown"
        elif suffix in TEXT_EXT:
            yield path, "text"

def read_text(path: Path) -> str:
    """Read a corpus file as UTF-8, replacing undecodable bytes."""
    with path.open("r", encoding="utf-8", errors="replace") as fh:
        return fh.read()

```

Why each decision

- **sorted(rglob)**: deterministic ordering makes indexing reproducible — chunk ids stay stable across runs, which debugging and incremental indexing (Volume 3, bonus 3) both depend on.
- **errors='replace'**: one undecodable byte must not kill a 5-minute indexing run; the replacement character occupies exactly one str position, so downstream character offsets remain consistent as long as every reader uses this same function. That last clause is the invariant: `read_text` is the ONLY way any part of the system opens a corpus file.
- **The 2 MB cap**: vLLM ships a few enormous generated files; their BM25 value is near zero and their chunking cost is real. Measure: with the cap, reference indexing runs in well under the 5-minute budget on a campus machine.
- **Paths are kept relative** (`data/raw/vllm-0.10.1/...`) from the moment of discovery, because `rglob` on a relative root yields relative paths. Convert with `.as_posix()` when serializing — on any OS, the grader compares forward-slash strings.

Watch out — Run discovery once and print the count by kind before building anything else. Expected order of magnitude for vLLM 0.10.1: a few thousand `.py` files, hundreds of `.md`, hundreds of small text files. Zero markdown files means your root path is wrong — catch it in minute one, not after wiring the whole pipeline.

CHAPTER 4

The Analyzer

Thirty lines that are worth twenty recall points. One function, imported by indexer and retriever alike.

Volume 1 chapter 8 designed this function stage by stage; here is the implementation, complete with the precompiled patterns and the keep-both-forms rule for identifiers:

src/analyzer.py — complete

```
"""Text analysis: raw text -> normalized index terms.

The SAME function must process corpus chunks (indexing) and
queries (retrieval). Import it; never reimplement it.
"""

import re
from typing import List

TOKEN_RE = re.compile(r"[A-Za-z0-9_]+")
CAMEL_RE = re.compile(
    r"(?<=[a-z0-9])(?=[A-Z])|(?<=[A-Z])(?=[A-Z][a-z])"
)

STOPWORDS = frozenset(
    "a an and are as at be by for from has have how i in is it"
    " its of on or that the this to was we what when where which"
    " will with you your".split()
)

def split_identifier(token: str) -> List[str]:
    """Split snake_case and camelCase into lowercase parts."""
    parts: List[str] = []
    for piece in token.split("_"):
        if piece:
            parts.extend(CAMEL_RE.split(piece))
    return [p.lower() for p in parts if p]

def analyze(text: str) -> List[str]:
    """Turn raw text into the list of terms to index/match.

    Emits, for each raw token, its lowercase form AND (when a
    compound identifier) its lowercase parts, so paraphrases
    and verbatim identifier mentions both match.
    """
    terms: List[str] = []
    for token in TOKEN_RE.findall(text):
        lower = token.lower()
        if lower not in STOPWORDS:
            terms.append(lower)
        parts = split_identifier(token)
        if len(parts) > 1:
            terms.extend(
                p for p in parts if p not in STOPWORDS
            )
```

```
)  
return terms
```

Reading the subtleties

- The intact lowercase token is emitted first, parts only when the split actually produced more than one piece — a plain word like “memory” is not duplicated (duplication would double its TF and distort BM25 saturation for ordinary prose).
- A compound like `gpu_memory_utilization` therefore contributes four terms: itself plus three parts. Its own TF stays 1; each part gets TF 1. BM25 sees a chunk that “contains” all four — precisely the multi-key behaviour Volume 1 argued for.
- The stoplist is deliberately small and lowercase-only; it filters both whole tokens and parts (splitting CamelCase can produce “the” from “TheClass” — filtered).
- No stemming in the reference build; Volume 3 chapter 3 shows how to add Snowball behind a flag and measure the delta. Adding it here would couple a dependency to un-measured benefit.

Note — Property worth a unit test: `analyze` is idempotent on its own output vocabulary (analyzing a term list's `join` returns the same multiset), and `analyze("") == []`. Both take four lines in `pytest` and freeze the analyzer's contract against future 'improvements' that would silently desynchronize an existing index.

CHAPTER 5

The Markdown / Text Chunker

Cut where the author already cut: headings, then paragraphs, then — only as a last resort — mid-text.

Both chunkers share one output type and one invariant (span $\leq$ max_chunk_size, offsets into the raw decoded text — Volume 1 ch. 9). Define the shared pieces first:

src/chunkers.py (part 1) — the shared Chunk type

```
"""Chunkers: file text -> list of Chunk with exact char spans."""
import ast
import re
from dataclasses import dataclass
from typing import List

@dataclass(frozen=True)
class Chunk:
    """A contiguous span of one file (end index exclusive)."""

    file_path: str
    start: int
    end: int
    text: str

HEADING_RE = re.compile(r"^{1,6}\s", re.MULTILINE)
FENCE_RE = re.compile(r"^\s*(```|~~~)")
```

A dataclass, not pydantic: this is an internal hot-path type created tens of thousands of times during indexing; the subject requires pydantic only for the *exchanged* data models, and explicitly exempts service internals. Say that sentence at the defense when asked.

The algorithm

Scan the file line by line, tracking code-fence state. A heading outside a fence closes the current section. Sections are then packed into chunks: whole paragraphs (blank-line separated) are accumulated greedily up to max_chunk_size; an oversized single paragraph falls back to a hard split with a small overlap:

src/chunkers.py (part 2) — markdown/text strategy, complete

```
def chunk_markdown(
    file_path: str, text: str, max_chunk_size: int = 2000
) -> List[Chunk]:
    """Chunk markdown/plain text along headings and paragraphs."""
    if not text:
        return []
    sections = _split_sections(text)
    chunks: List[Chunk] = []
    for sec_start, sec_end in sections:
```

```

        chunks.extend(
            _pack_span(file_path, text, sec_start, sec_end,
                       max_chunk_size)
        )
    return chunks

def _split_sections(text: str) -> List[tuple[int, int]]:
    """Return (start, end) spans split at headings outside fences."""
    bounds: List[int] = [0]
    offset = 0
    in_fence = False
    for line in text.splitlines(keepends=True):
        if FENCE_RE.match(line):
            in_fence = not in_fence
        elif not in_fence and HEADING_RE.match(line) and offset:
            bounds.append(offset)
        offset += len(line)
    bounds.append(len(text))
    return [
        (bounds[i], bounds[i + 1])
        for i in range(len(bounds) - 1)
        if bounds[i + 1] > bounds[i]
    ]

def _pack_span(
    file_path: str, text: str, start: int, end: int, limit: int
) -> List[Chunk]:
    """Pack [start, end) into chunks of at most limit chars.

    Prefers paragraph boundaries; falls back to hard cuts with
    a 200-character overlap for oversized paragraphs.
    """
    chunks: List[Chunk] = []
    paragraphs = _paragraph_spans(text, start, end)
    cur_start = start
    cur_end = start
    for p_start, p_end in paragraphs:
        if p_end - cur_start <= limit:
            cur_end = p_end
            continue
        if cur_end > cur_start:
            chunks.append(_make(file_path, text, cur_start, cur_end))
        if p_end - p_start <= limit:
            cur_start, cur_end = p_start, p_end
        else:
            chunks.extend(
                _hard_split(file_path, text, p_start, p_end, limit)
            )
            cur_start = cur_end = p_end
    if cur_end > cur_start:
        chunks.append(_make(file_path, text, cur_start, cur_end))
    return chunks

def _paragraph_spans(
    text: str, start: int, end: int
) -> List[tuple[int, int]]:
    """Spans of blank-line-separated blocks inside [start, end)."""
    spans: List[tuple[int, int]] = []

```

```

    block_start = start
    offset = start
    for line in text[start:end].splitlines(keepends=True):
        if not line.strip():
            if offset > block_start:
                spans.append((block_start, offset))
                block_start = offset + len(line)
            offset += len(line)
        if end > block_start:
            spans.append((block_start, end))
    return spans

def _hard_split(
    file_path: str, text: str, start: int, end: int, limit: int
) -> List[Chunk]:
    """Last-resort fixed windows with 200-char overlap."""
    step = max(limit - 200, 1)
    out: List[Chunk] = []
    pos = start
    while pos < end:
        out.append(
            _make(file_path, text, pos, min(pos + limit, end))
        )
        pos += step
    return out

def _make(file_path: str, text: str, start: int, end: int) -> Chunk:
    """Build a Chunk; the single place the invariant is asserted."""
    assert 0 < end - start <= 2000, (file_path, start, end)
    return Chunk(file_path, start, end, text[start:end])

```

Design notes

- `splitlines(keepends=True)` does all offset bookkeeping: summing line lengths reproduces exact character positions, including `\r\n` files, with no separate index arithmetic to get wrong.
- The heading keeps its section (bounds are *before* the heading line): a chunk starts with its title — Volume 1's argument that heading words are query words, embodied in one inequality (``and offset``).
- The 200-char overlap in `_hard_split` exists only on the fallback path; structural cuts don't need it. The step guard `max(limit-200, 1)` keeps tiny `max_chunk_size` values (a reviewer WILL try `--max_chunk_size 50`) from looping forever.
- `_make` asserts against 2000 as an absolute constant, not against the configurable limit: even if someone passes `--max_chunk_size 5000`, the moulinette's hard ceiling is enforced at the last constructor. Clamp the CLI argument too (chapter 9), but defense in depth is free here.

CHAPTER 6

The Python AST Chunker

Functions and classes are the meaning units of code. Cut there, and a method arrives in the context window whole.

The Python strategy parses each file and emits one chunk per top-level definition, descending into classes whose bodies exceed the limit, and gathering interstitial module-level code. Line numbers from ast become character offsets via a line-offset table:

src/chunkers.py (part 3) — Python AST strategy, complete

```
def _line_offsets(text: str) -> List[int]:
    """offsets[i] = char index of the start of line i+1 (1-based)."""
    offsets = [0]
    for line in text.splitlines(keepends=True):
        offsets.append(offsets[-1] + len(line))
    return offsets

def chunk_python(
    file_path: str, text: str, max_chunk_size: int = 2000
) -> List[Chunk]:
    """Chunk Python source along top-level defs and classes."""
    if not text:
        return []
    try:
        tree = ast.parse(text)
    except (SyntaxError, ValueError):
        return chunk_markdown(file_path, text, max_chunk_size)
    offsets = _line_offsets(text)
    spans = _definition_spans(tree, offsets, text, max_chunk_size)
    return _spans_to_chunks(
        file_path, text, spans, max_chunk_size
    )

def _node_span(
    node: ast.stmt, offsets: List[int]
) -> tuple[int, int]:
    """Char span of a statement, decorators included."""
    start_line = node.lineno
    deco = getattr(node, "decorator_list", None)
    if deco:
        start_line = min(d.lineno for d in deco)
    end_line = node.end_lineno or start_line
    return offsets[start_line - 1], offsets[end_line]

def _definition_spans(
    tree: ast.Module, offsets: List[int], text: str, limit: int
) -> List[tuple[int, int]]:
    """Spans for top-level defs; large classes split per method."""
    spans: List[tuple[int, int]] = []
    prev_end = 0
```

```

for node in tree.body:
    start, end = _node_span(node, offsets)
    if start > prev_end:
        spans.append((prev_end, start)) # interstitial code
    big_class = (
        isinstance(node, ast.ClassDef)
        and end - start > limit
    )
    if big_class:
        spans.extend(
            _class_member_spans(node, offsets, limit)
        )
    else:
        spans.append((start, end))
    prev_end = end
if len(text) > prev_end:
    spans.append((prev_end, len(text)))
return [(s, e) for s, e in spans if e > s]

def _class_member_spans(
    node: ast.ClassDef, offsets: List[int], limit: int
) -> List[tuple[int, int]]:
    """Header span plus one span per method/member."""
    class_start, class_end = _node_span(node, offsets)
    if not node.body:
        return [(class_start, class_end)]
    first_member_start, _ = _node_span(node.body[0], offsets)
    spans = [(class_start, first_member_start)] # class header
    prev = first_member_start
    for member in node.body:
        _, m_end = _node_span(member, offsets)
        spans.append((prev, m_end))
        prev = m_end
    if class_end > prev:
        spans.append((prev, class_end))
    return spans

def _spans_to_chunks(
    file_path: str, text: str,
    spans: List[tuple[int, int]], limit: int,
) -> List[Chunk]:
    """Merge small spans, hard-split oversized ones, build Chunks."""
    chunks: List[Chunk] = []
    cur_start = cur_end = None # type: ignore[assignment]
    for start, end in spans:
        if cur_start is None:
            cur_start, cur_end = start, end
        elif end - cur_start <= limit:
            cur_end = end
        else:
            chunks.extend(
                _flush(file_path, text, cur_start, cur_end, limit)
            )
            cur_start, cur_end = start, end
    if cur_start is not None:
        chunks.extend(
            _flush(file_path, text, cur_start, cur_end, limit)
        )
    return chunks

```

```
def _flush(
    file_path: str, text: str, start: int, end: int, limit: int
) -> List[Chunk]:
    """One span -> chunk(s), respecting the size limit."""
    if end - start <= limit:
        stripped = text[start:end]
        if not stripped.strip():
            return []
        return [_make(file_path, text, start, end)]
    return _hard_split(file_path, text, start, end, limit)
```

Why it is written this way

- **SyntaxError falls back to the text chunker**, never to skipping the file: a Python 2 relic or Jinja-templated .py still carries indexable vocabulary, and a skipped file is an unreachable ground-truth source.
- **Decorators are included** in a definition's span (min over decorator_list linenos): @torch.no_grad() above a method is both real vocabulary and real context.
- **Method chunks keep continuity**: _class_member_spans emits the class header (class line + docstring up to the first member) as its own small span, and _spans_to_chunks' greedy merge then glues it to the first method(s) — so a retrieved method usually arrives prefixed by its class line, giving Qwen the “class Scheduler:” context Volume 1 promised.
- **Small-span merging** also stops one-line module constants from becoming hundreds of confetti chunks that pollute top-k with near-empty matches.
- **Empty/whitespace spans are dropped at _flush**: they can match nothing and would only dilute avgdl, subtly distorting BM25 length normalization.

Note — The merge means chunks are not 1:1 with functions — they are *sequences of adjacent definitions* packed to the limit. That is intentional: recall rewards containing the true span (IoU only 0.05), and packed chunks near the ceiling contain more candidate spans. If you later experiment with unmerged per-function chunks, measure — Volume 3's harness makes the comparison a one-flag affair.

Watch out — ast line numbers are 1-based and end_lineno is inclusive; the offsets table converts via offsets[start_line-1] and offsets[end_line]. Off by one here and every code chunk shifts a line — recall drops a few points and NOTHING crashes. Unit-test _node_span against a file you wrote by hand, character-counted in an editor.

CHAPTER 7

Building and Persisting the BM25 Index

Everything retrieval will ever know is computed here, once, and written under data/processed/.

The index is four aligned structures: the chunk table (paths, spans, texts), per-chunk term-frequency maps folded into an inverted index, per-chunk lengths, and corpus statistics (N, avgdl, document frequencies). Volume 1 chapters 5-7 derived why each exists. The builder:

src/indexer.py (part 2) — build, save, load

```
"""Index construction (continues src/indexer.py)."""
import json
import math
import pickle
import time
from collections import Counter
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List

from tqdm import tqdm

from src.analyzer import analyze
from src.chunkers import Chunk, chunk_markdown, chunk_python

PROCESSED_DIR = Path("data/processed")
INDEX_FILE = PROCESSED_DIR / "bm25_index.pkl"
META_FILE = PROCESSED_DIR / "index_meta.json"

@dataclass
class Bm25Index:
    """Everything needed to score any query against the corpus."""

    chunks: List[Chunk] = field(default_factory=list)
    postings: Dict[str, Dict[int, int]] = field(default_factory=dict)
    doc_len: List[int] = field(default_factory=list)
    avgdl: float = 0.0

    @property
    def n_chunks(self) -> int:
        """Number of chunks in the corpus."""
        return len(self.chunks)

def build_index(
    raw_root: Path = RAW_ROOT, max_chunk_size: int = 2000
) -> Bm25Index:
    """Chunk and analyze the whole corpus into a Bm25Index."""
    index = Bm25Index()
    files = list(iter_corpus_files(raw_root))
    for path, kind in tqdm(files, desc="Indexing", unit="file"):
        text = read_text(path)
```

```

    rel = path.as_posix()
    if kind == "python":
        chunks = chunk_python(rel, text, max_chunk_size)
    else:
        chunks = chunk_markdown(rel, text, max_chunk_size)
    for chunk in chunks:
        _add_chunk(index, chunk)
    total = sum(index.doc_len)
    index.avgdl = total / index.n_chunks if index.n_chunks else 0.0
    return index

def _add_chunk(index: Bm25Index, chunk: Chunk) -> None:
    """Analyze one chunk and fold it into the statistics."""
    chunk_id = len(index.chunks)
    terms = analyze(chunk.text)
    index.chunks.append(chunk)
    index.doc_len.append(len(terms))
    for term, tf in Counter(terms).items():
        index.postings.setdefault(term, {})[chunk_id] = tf

def save_index(index: Bm25Index) -> None:
    """Persist the index and a small human-readable meta file."""
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    with INDEX_FILE.open("wb") as fh:
        pickle.dump(index, fh, protocol=pickle.HIGHEST_PROTOCOL)
    meta = {
        "n_chunks": index.n_chunks,
        "n_terms": len(index.postings),
        "avgdl": round(index.avgdl, 2),
        "built_at": time.strftime("%Y-%m-%d %H:%M:%S"),
    }
    with META_FILE.open("w", encoding="utf-8") as fh:
        json.dump(meta, fh, indent=2)

def load_index() -> Bm25Index:
    """Load the pickled index; raise a clear error if absent."""
    if not INDEX_FILE.exists():
        raise FileNotFoundError(
            "No index found under data/processed/. "
            "Run: uv run python -m src index"
        )
    with INDEX_FILE.open("rb") as fh:
        index = pickle.load(fh)
    if not isinstance(index, Bm25Index):
        raise TypeError("Corrupt index file; re-run index.")
    return index

```

Choices you should be able to defend

- **Pickle for the index:** it round-trips the dataclass (including chunk texts) in one call and loads a full corpus index in well under a second — comfortably inside the 90-second batch budget. The JSON meta file exists so a human (you, at the defense) can cat something meaningful. If pickle's fragility bothers you, JSON-serialize the same four structures — slower to load, equally valid; what matters is a single load per process, never per query.

- **Chunk texts are stored in the index.** Retrieval does not strictly need them (offsets suffice), but the answer stage does, and re-reading files at answer time would re-open the encoding/offset consistency risk for zero gain. The full index for this corpus is tens of MB — cheap.
- **tqdm wraps the file loop**, satisfying the progress-bar requirement where users actually wait. Wrap dataset loops in `search_dataset` and `answer_dataset` the same way (chapter 9).
- **Determinism:** chunk ids are assignment order, which is sorted file order, which is stable — the property Volume 3's incremental-indexing bonus builds on.

Note — Expected scale on vLLM 0.10.1 with defaults (order of magnitude): 20,000-40,000 chunks, 100,000-200,000 distinct terms, index build 1-3 minutes on a campus machine, pickle around 50-100 MB. If your numbers differ wildly, chapter 3's discovery filters are the first suspect — print the per-kind file counts again.

CHAPTER 8

The Retriever

Sixty lines that the entire grade flows through: BM25 scoring, top-k selection, and the exact output contract.

src/retriever.py — complete

```

"""BM25 retrieval over the persisted index."""
import heapq
import math
from typing import List, Tuple

from src.analyzer import analyze
from src.indexer import Bm25Index
from src.models import MinimalSource

K1 = 1.5
B = 0.75

def bm25_scores(index: Bm25Index, query: str) -> List[Tuple[int, float]]:
    """Score all chunks touched by the query's terms.

    Returns (chunk_id, score) pairs, unsorted, only for chunks
    with at least one matching term.
    """
    terms = analyze(query)
    if not terms or not index.n_chunks:
        return []
    n = index.n_chunks
    scores: dict[int, float] = {}
    for term in set(terms):
        postings = index.postings.get(term)
        if not postings:
            continue
        df = len(postings)
        idf = math.log((n - df + 0.5) / (df + 0.5) + 1.0)
        for chunk_id, tf in postings.items():
            norm = 1.0 - B + B * (
                index.doc_len[chunk_id] / index.avgdl
            )
            gain = tf * (K1 + 1.0) / (tf + K1 * norm)
            scores[chunk_id] = scores.get(chunk_id, 0.0) + idf * gain
    return list(scores.items())

def search(
    index: Bm25Index, query: str, k: int = 10
) -> List[MinimalSource]:
    """Top-k sources for one query, best first."""
    if k <= 0:
        return []
    scored = bm25_scores(index, query)
    top = heapq.nlargest(k, scored, key=lambda pair: pair[1])
    return [
        MinimalSource(

```

```

        file_path=index.chunks[cid].file_path,
        first_character_index=index.chunks[cid].start,
        last_character_index=index.chunks[cid].end,
    )
    for cid, _score in top
]
```

Line-by-line commentary

- **set(terms)**: query-side term repetition is deliberately ignored (“scheduler scheduler scheduler” should not triple-weight the term). Standard BM25 practice; if you prefer query-TF weighting, that is the BM25 qtf extension — know which you implemented.
- **Accumulator dict, not a matrix**: only chunks sharing at least one term with the query are touched — Volume 1’s inverted-index argument. Typical query: a few thousand dict operations, well under a millisecond of Python.
- **heapq.nlargest(k, ...)** is $O(M \log k)$ over M scored chunks versus $O(M \log M)$ for a full sort — a micro-optimization that also reads as intent: we want exactly k .
- **$k \leq 0$ returns []** — the subject’s degenerate-input list explicitly includes $k=0$; an empty result list is valid StudentSearchResults content, and nothing crashes.
- **The MinimalSource is built from the chunk’s stored span** — the only place retrieval output is created, so the ≤ 2000 invariant asserted at chunk construction is automatically carried to the wire format.

Batch search over a dataset

Batch mode: one index load, N cheap queries — the 90 s budget is safe

```

def search_dataset_results(
    index: Bm25Index, dataset: RagDataset, k: int
) -> StudentSearchResults:
    """Run search over every question of a dataset."""
    results = [
        MinimalSearchResults(
            question_id=q.question_id,
            question=q.question,
            retrieved_sources=search(index, q.question, k),
        )
        for q in tqdm(
            dataset.rag_questions, desc="Searching", unit="q"
        )
    ]
    return StudentSearchResults(search_results=results, k=k)
```

Measure it once and write the number down for the defense: 200 questions against the full index typically complete in 1-3 seconds — a 30-60x margin against the budget. The margin is the design: if you ever add reranking or dense fusion (bonuses), you know exactly how much time you may spend per question.

CHAPTER 9

The CLI with Python Fire

Six commands, one class. Fire maps methods to subcommands and keyword arguments to flags — the CLI writes itself.

Fire exposes a class's public methods as subcommands: `fire.Fire(RagCli)` makes `uv run python -m src search 'query' --k 5` call `RagCli().search('query', k=5)`. Docstrings become `--help` text. The full command surface required by the subject, in one class:

src/cli.py — all six mandatory commands

```
"""Fire CLI: the six mandatory commands."""
from pathlib import Path
from typing import Optional

from src import indexer, retriever
from src.models import (
    RagDataset, StudentSearchResults, load_model, save_model,
)

class RagCli:
    """RAG against the machine - command-line interface."""

    def index(self, max_chunk_size: int = 2000) -> str:
        """Ingest data/raw/ and build the index in data/processed/.

        Args:
            max_chunk_size: maximum chunk width in characters
                           (values above 2000 are clamped: the evaluator
                           rejects longer sources).
        """
        size = max(1, min(int(max_chunk_size), 2000))
        idx = indexer.build_index(max_chunk_size=size)
        indexer.save_index(idx)
        return (
            f"Ingestion complete! {idx.n_chunks} chunks "
            f"indexed under data/processed/"
        )

    def search(self, query: str = "", k: int = 10) -> list[dict]:
        """Return the top-k sources for a single query."""
        idx = indexer.load_index()
        sources = retriever.search(idx, str(query), int(k))
        return [s.model_dump() for s in sources]

    def search_dataset(
        self,
        dataset_path: str,
        k: int = 10,
        save_directory: str = "data/output/search_results",
    ) -> str:
        """Search every question of a dataset; write the JSON."""
        dataset = load_model(Path(dataset_path), RagDataset)
```

```

        idx = indexer.load_index()
        results = retriever.search_dataset_results(
            idx, dataset, int(k)
        )
        out = Path(save_directory) / Path(dataset_path).name
        save_model(out, results)
        return f"Saved student_search_results to {out}"

def answer(self, query: str = "", k: int = 10) -> str:
    """Answer a single query from retrieved context."""
    from src import generator # heavy import, deferred
    idx = indexer.load_index()
    sources = retriever.search(idx, str(query), int(k))
    return generator.answer_question(idx, str(query), sources)

def answer_dataset(
    self,
    student_search_results_path: str,
    save_directory: str = (
        "data/output/search_results_and_answer"
    ),
) -> str:
    """Generate answers for previously saved search results."""
    from src import generator
    results = load_model(
        Path(student_search_results_path), StudentSearchResults
    )
    idx = indexer.load_index()
    answered = generator.answer_dataset_results(idx, results)
    out = (
        Path(save_directory)
        / Path(student_search_results_path).name
    )
    save_model(out, answered)
    return f"Saved student_search_results_and_answer to {out}"

def evaluate(
    self,
    student_search_results_path: str,
    dataset_path: str,
) -> dict[str, float]:
    """Report recall@k against a ground-truth dataset."""
    from src.evaluation import evaluate_results
    return evaluate_results(
        Path(student_search_results_path), Path(dataset_path)
    )

```

Fire behaviours worth knowing before the defense

- Fire performs its own string-to-type coercion from the command line, and an argument like `--k banana` arrives as the string `'banana'`. The explicit `int(k)/str(query)` casts inside each method are the cheap, visible line of defense; chapter 10 wraps the remaining failure modes.
- Returning a value (rather than printing) lets Fire render it — returning the list of dicts from search gives readable output for free, and returning strings gives the exact “Ingestion complete!” lines the subject's walkthrough shows.

- Heavy imports (torch, transformers) are deferred into the two answer commands: index/search/evaluate stay snappy and work on machines where torch is missing or broken — and the reviewer's first command is never greeted by a 10-second import.
- All paths are arguments with subject-compliant defaults; nothing is hard-coded. The evaluator will point `--save_directory` at scoped folders (`.../UnansweredQuestions`) exactly as the walkthrough shows — your code already accepts it.

CHAPTER 10

Robustness: Never Crash

“The CLI is tested with such edge cases and must never crash with an unhandled traceback.” That sentence is a grading rubric. Engineer for it.

The subject enumerates the attack surface: empty query, nonsensical query, $k=0$, missing files, malformed JSON. The pattern that handles all of them without smearing try/except through every function: let the domain code raise precise exceptions, and convert them to clean messages at ONE boundary — the CLI entry point.

src/__main__.py (final form) — the single error boundary

```
"""Entry point with the error boundary (final __main__.py)."""
import sys

import fire
from pydantic import ValidationError

from src.cli import RagCli

def main() -> None:
    """Run the CLI; translate known failures into clean exits."""
    try:
        fire.Fire(RagCli)
    except FileNotFoundError as exc:
        _fail(f"File not found: {exc}")
    except ValidationError as exc:
        _fail(f"Invalid input file format:\n{exc}")
    except (ValueError, TypeError) as exc:
        _fail(f"Invalid argument: {exc}")
    except KeyboardInterrupt:
        _fail("Interrupted.", code=130)
    except OSError as exc:
        _fail(f"I/O error: {exc}")

def _fail(message: str, code: int = 1) -> None:
    """Print an error to stderr and exit non-zero, no traceback."""
    print(f"error: {message}", file=sys.stderr)
    raise SystemExit(code)
```

Walking the subject's edge-case list

Input	Path through the system	Observed behaviour
empty query "	analyze("") -> [] -> bm25_scores returns [] -> search returns []	empty result list, exit 0
nonsensical query 'xqzt vprw'	terms exist, no postings match	empty or low-score results, exit 0
--k 0	search short-circuits $k \leq 0$	empty list, exit 0

Input	Path through the system	Observed behaviour
--k -3	same guard	empty list, exit 0
missing dataset file	load_model raises FileNotFoundError	'error: File not found: ...', exit 1
malformed JSON	model_validate_json raises ValidationError	'error: Invalid input file format: ...', exit 1
index not built yet	load_index raises FileNotFoundError with the fix in the message	'error: ... Run: uv run python -m src index', exit 1
--max_chunk_size 999999	clamped to 2000 in cli.index	valid index, no oversized sources
--max_chunk_size 0	clamped to 1; hard-split step guard prevents infinite loops	slow but valid; degenerate by user request

Two disciplines complete the picture. First, **context managers everywhere**: every `open()` in this volume is a `with`-block — the subject names resource management explicitly, and the reviewer may `grep` for bare `open`. Second, **messages must name the fix**, not just the failure: compare “`KeyError: 'bm25_index'`” with “No index found under `data/processed/`. Run: `uv run python -m src index`”. The second is what a grader under time pressure remembers as “robust”.

Note — Test the boundary the honest way: run each row of the table above against your real CLI and watch the exit codes (`echo $?`). Five minutes, scriptable, and it is precisely what the evaluator will do to you.

CHAPTER 11

Answer Generation with Qwen3-0.6B

Load once, prompt carefully, decode fast, post-process defensively.

Volume 1 chapter 12 designed the prompt and the policies (top-5 context, thinking off, escape hatch, 256-token cap). Implementation notes first, then the code: the model loads once per process (module-level cache), context is assembled under an explicit character budget, and the output is stripped of any thinking residue before it touches a pydantic model.

src/generator.py (part 1) — model, context, one generation

```

"""Answer generation with Qwen/Qwen3-0.6B (src/generator.py)."""
from typing import List, Optional, Tuple

from tqdm import tqdm

from src.indexer import Bm25Index
from src.models import (
    MinimalAnswer, MinimalSource, StudentSearchResults,
    StudentSearchResultsAndAnswer,
)

MODEL_NAME = "Qwen/Qwen3-0.6B"
CONTEXT_BUDGET_CHARS = 8000
MAX_SOURCES_IN_PROMPT = 5
MAX_NEW_TOKENS = 256

SYSTEM_PROMPT = (
    "You are a precise technical assistant answering questions"
    " about the vLLM codebase. Answer ONLY from the provided"
    " context snippets. If the context does not contain the"
    " answer, say so briefly. Cite file paths when relevant."
    " Be concise: 2-6 sentences."
)

_MODEL_CACHE: Optional[Tuple[object, object]] = None

def _load_model() -> Tuple[object, object]:
    """Load tokenizer and model once per process."""
    global _MODEL_CACHE
    if _MODEL_CACHE is None:
        from transformers import (
            AutoModelForCausalLM, AutoTokenizer,
        )
        tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        model = AutoModelForCausalLM.from_pretrained(
            MODEL_NAME, torch_dtype="auto"
        )
        _MODEL_CACHE = (tokenizer, model)
    return _MODEL_CACHE

def _chunk_text(index: Bm25Index, source: MinimalSource) -> str:

```

```

"""Recover a source's text from the index chunk table."""
for chunk in index.chunks:
    if (
        chunk.file_path == source.file_path
        and chunk.start == source.first_character_index
        and chunk.end == source.last_character_index
    ):
        return chunk.text
return "" # source not from this index: skip silently

def _build_context(
    index: Bm25Index, sources: List[MinimalSource]
) -> str:
    """Label and concatenate top sources under the budget."""
    blocks: List[str] = []
    used = 0
    for rank, source in enumerate(
        sources[:MAX_SOURCES_IN_PROMPT], start=1
    ):
        text = _chunk_text(index, source)
        if not text:
            continue
        block = (
            f"[Source {rank}] {source.file_path}"
            f" (chars {source.first_character_index}"
            f"-{source.last_character_index})\n{text}"
        )
        if used + len(block) > CONTEXT_BUDGET_CHARS:
            break
        blocks.append(block)
        used += len(block)
    return "\n\n".join(blocks)

def generate_answer(question: str, context: str) -> str:
    """One grounded generation, thinking disabled."""
    if not context.strip():
        return (
            "No relevant sources were retrieved for this"
            " question, so no grounded answer can be given."
        )
    tokenizer, model = _load_model()
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {
            "role": "user",
            "content": (
                f"Context: \n{context}\n\n"
                f"Question: {question}"
            )
        },
    ]
    prompt = tokenizer.apply_chat_template( # type: ignore[attr-defined]
        messages,
        tokenize=False,
        add_generation_prompt=True,
        enable_thinking=False,
    )
    inputs = tokenizer( # type: ignore[operator]
        prompt, return_tensors="pt"

```

```

    )
    output = model.generate( # type: ignore[attr-defined]
        **inputs,
        max_new_tokens=MAX_NEW_TOKENS,
        do_sample=True,
        temperature=0.7,
        top_p=0.8,
        top_k=20,
    )
    new_tokens = output[0][inputs["input_ids"].shape[1]:]
    text = tokenizer.decode( # type: ignore[attr-defined]
        new_tokens, skip_special_tokens=True
    )
    return _clean(text)

def _clean(text: str) -> str:
    """Strip any thinking residue and tidy whitespace."""
    if "</think>" in text:
        text = text.split("</think>", 1)[1]
    return " ".join(text.split()).strip()

```

src/generator.py (part 2) — dataset mode

```

def answer_question(
    index: Bm25Index, question: str, sources: List[MinimalSource]
) -> str:
    """Single-question entry used by the answer command."""
    context = _build_context(index, sources)
    return generate_answer(question, context)

def answer_dataset_results(
    index: Bm25Index, results: StudentSearchResults
) -> StudentSearchResultsAndAnswer:
    """Answer every question of saved search results."""
    answered: List[MinimalAnswer] = []
    for res in tqdm(
        results.search_results, desc="Answering", unit="q"
    ):
        answer = answer_question(
            index, res.question, res.retrieved_sources
        )
        answered.append(
            MinimalAnswer(
                question_id=res.question_id,
                question=res.question,
                retrieved_sources=res.retrieved_sources,
                answer=answer,
            )
        )
    return StudentSearchResultsAndAnswer(
        search_results=answered, k=results.k
    )

```

Operational notes

- **First run downloads ~1.5 GB** (weights + tokenizer) into the HuggingFace cache; the subject explicitly warns to build from a location with disk space. Do the first download before the defense day.

- **CPU timing:** expect a few seconds per answer with 256 new tokens. For a 100-question dataset that is minutes, not hours — but test on campus-grade hardware, not your laptop's GPU, because that is where the demo happens.
- **_chunk_text scans linearly;** for dataset-scale answering, build a dict keyed by (path, start, end) once if profiling shows it matters. It is $O(n_chunks)$ per source but n is small against generation cost — measure before optimizing, and be able to say you did.
- **The empty-context guard** means a question whose retrieval came back empty still produces a valid MinimalAnswer with an honest string — the degenerate path stays crash-free end to end.

Watch out — If answers come out prefixed with deliberation or oddly slow, `enable_thinking=False` did not take: check transformers ≥ 4.51 and that the flag is passed to `apply_chat_template` (not to `generate`). The `_clean` fallback keeps output usable either way — belt and suspenders.

CHAPTER 12

Your Own evaluate, and the Full Pipeline Run

Reproduce the grader's arithmetic, then run the four commands exactly as the exam scripts will.

Your evaluate command exists so tuning never waits on the moulinette. It must reproduce the metric exactly — same-file + IoU $\geq$ 0.05, macro recall — or it will mislead you (Volume 1 ch. 11 derived every line):

src/evaluation.py — complete

```
"""Self-evaluation: recall@k identical to the moulinette."""
from pathlib import Path
from typing import Dict, List

from src.models import (
    AnsweredQuestion, MinimalSource, RagDataset,
    StudentSearchResults, load_model,
)

IOU_THRESHOLD = 0.05
KS = (1, 3, 5, 10)

def _iou(a: MinimalSource, b: MinimalSource) -> float:
    """Interval IoU of two sources (0 if different files)."""
    if a.file_path != b.file_path:
        return 0.0
    inter = max(
        0,
        min(a.last_character_index, b.last_character_index)
        - max(a.first_character_index, b.first_character_index),
    )
    union = (
        (a.last_character_index - a.first_character_index)
        + (b.last_character_index - b.first_character_index)
        - inter
    )
    return inter / union if union > 0 else 0.0

def _recall_at(
    retrieved: List[MinimalSource],
    truth: List[MinimalSource],
    k: int,
) -> float:
    """Share of truth sources matched within the top k."""
    if not truth:
        return 1.0
    top = retrieved[:k]
    found = sum(
        1
```

```

        for g in truth
            if any(_iou(r, g) >= IOU_THRESHOLD for r in top)
        )
    return found / len(truth)

def evaluate_results(
    results_path: Path, dataset_path: Path
) -> Dict[str, float]:
    """Macro recall@k of saved results vs ground truth."""
    results = load_model(results_path, StudentSearchResults)
    dataset = load_model(dataset_path, RagDataset)
    truth = {
        q.question_id: q.sources
        for q in dataset.rag_questions
        if isinstance(q, AnsweredQuestion)
    }
    per_k: Dict[str, List[float]] = {f"recall@{k}": [] for k in KS}
    for res in results.search_results:
        gt = truth.get(res.question_id)
        if gt is None:
            continue
        for k in KS:
            per_k[f"recall@{k}"].append(
                _recall_at(res.retrieved_sources, gt, k)
            )
    return {
        name: round(sum(vals) / len(vals), 3) if vals else 0.0
        for name, vals in per_k.items()
    }

```

Watch out — Your solution must never import or call the *moulinette* — the subject says so explicitly. This module REPRODUCES its arithmetic from the spec; it shares no code with it. Keep the executable out of your `src/` imports entirely.

The four-command run, exactly as the defense will execute it

The end-to-end walkthrough from the subject, runnable verbatim

```

# 1. index the corpus (once)
uv run python -m src index --max_chunk_size 2000

# 2. search a whole dataset (scope the save dir by dataset!)
uv run python -m src search_dataset \
    --dataset_path data/datasets/UnansweredQuestions/dataset_docs_public.json \
    --k 10 \
    --save_directory data/output/search_results/UnansweredQuestions

# 3. score with the moulinette (student results first!)
./moulinette evaluate_student_search_results \
    data/output/search_results/UnansweredQuestions/dataset_docs_public.json \
    data/datasets/AnsweredQuestions/dataset_docs_public.json \
    --k 10 --max_context_length 2000

# 4. generate answers from the saved search results
uv run python -m src answer_dataset \
    --student_search_results_path \
        data/output/search_results/UnansweredQuestions/dataset_docs_public.json \
    --save_directory data/output/search_results_and_answer/UnansweredQuestions

```

Three traps hidden in these four commands, all named by the subject: (1) **scope save directories by dataset** — the public docs and code datasets share file names, and an unscoped directory silently overwrites one with the other; (2) **argument order for the moulinette** is student results first, ground truth second; (3) the moulinette binary ships as moulinette-ubuntu / moulinette-fedora and must be renamed (and chmod +x'd) to ./moulinette. Rehearse the full sequence cold before the defense — it is the first thing the evaluator runs.

CHAPTER 13

Anatomy of the Datasets

Four JSON files decide your grade. Spend an hour reading them before you spend a week tuning against them.

The repository ships four public datasets under `data/datasets/`: docs and code variants, each in an `UnansweredQuestions` form (questions only — what `search_dataset` consumes) and an `AnsweredQuestions` form (questions + reference answers + ground-truth sources — what the `moulinette` scores against). The defense adds private counterparts drawn from the same distribution; the public files are your only window into it, so learn their shape thoroughly and honestly (tune on them, but remember the private set punishes overfitting to specific questions).

Reading them programmatically

Your own models load the files directly — which is itself a test of your models. A ten-line exploration script pays for itself immediately:

explore_dataset.py — run it against all four public files

```
import collections
from pathlib import Path

from src.models import AnsweredQuestion, RagDataset, load_model

ds = load_model(
    Path("data/datasets/AnsweredQuestions/dataset_docs_public.json"),
    RagDataset,
)
qs = [q for q in ds.rag_questions if isinstance(q, AnsweredQuestion)]
print("questions:", len(qs))
print("sources/question:", collections.Counter(
    len(q.sources) for q in qs
))
widths = [
    s.last_character_index - s.first_character_index
    for q in qs for s in q.sources
]
print("span widths: min", min(widths), "max", max(widths))
print("top source dirs:", collections.Counter(
    "/" + s.file_path.split("/")[4]
    for q in qs for s in q.sources
).most_common(8))
```

What this tells you, and why each number changes your engineering:

- **Questions per dataset** calibrates your batch timings (the 90 s budget is for 200 questions — know how close the real files come).
- **Sources per question**: the distribution of $|G(q)|$ explains your recall staircase (Volume 1 ch. 11 — multi-source questions cap recall at low k) and tells you how much diversity your top- k needs.

- **Span widths** reveal the ground truth's own chunking convention; if reference spans cluster near 300-1500 characters, near-ceiling chunks with a low IoU bar will cover them comfortably — the geometry argument for generous chunks, now with your own numbers in it.
- **Top source directories** show where answers live per dataset: docs questions concentrate under docs/, code questions under vllm/. This justifies per-corpus tuning (Volume 3) and tells you instantly whether your discovery filters cover the ground truth's territory — if a ground-truth path is a file your walker skips, that question is lost before you begin.

The question style

Read twenty questions aloud. The docs set asks user-manual questions (“How do I enable prefix caching?”, “Which quantization methods are supported?”) whose vocabulary maps well onto Markdown headings — this is why the docs threshold is 80%. The code set asks implementation-shaped questions (“What does the scheduler do when a request exceeds the token budget?”) whose vocabulary is identifiers and half-remembered phrases — the analyzer's identifier splitting and the AST chunker carry these, and 50% acknowledges the difficulty. When Volume 3's triage sorts your failures, you will recognize these two families immediately.

Watch out — Never hand-edit the dataset files, and never let output files land in data/datasets/. One overwritten ground-truth file makes every later measurement a lie, and git will not save you on files the subject told you not to commit. Treat the directory as read-only from day one (chmod -w if you don't trust yourself).

A hand-written results file demystifies the grader

Before your pipeline exists, you can already pass a file through the moulinette: write a StudentSearchResults JSON by hand with one question copied from a public dataset and its own ground-truth source as the single retrieved source, then run `evaluate_student_search_results`. You will see validation pass, `recall@1 = 1.0` for that question, and the exact output format — in five minutes, on day one. Students who do this never fear the grader again; students who don't, debug their first real run against an unfamiliar tool under pressure. The moulinette is also your format oracle: whenever you are unsure whether an output tweak is legal, ask it, not a guess.

CHAPTER 14

How a Query Flows: A Guided Code Reading

One command, every function it touches, in order — the reading you rehearse before someone asks you to explain your own system.

The single most common defense request is “walk me through what happens when I run this”. Here is the complete call graph for the two commands that matter most, at the level of detail an evaluator expects. Rehearse it with the code open.

uv run python -m src index --max_chunk_size 2000

1. Python resolves `-m src` to `src/__main__.py`; `main()` calls `fire.Fire(RagCli)`; `Fire` parses the argv, instantiates `RagCli`, and invokes `.index(max_chunk_size=2000)`.
2. `cli.index` clamps the size (`max(1, min(size, 2000))`) — the defensive line that makes `--max_chunk_size 999999` harmless — and calls `indexer.build_index`.
3. `build_index` lists `iter_corpus_files()`: sorted rglob under `data/raw`, skip-dirs and size filters applied, each file tagged python / markdown / text (chapter 3's policy table).
4. Per file: `read_text` decodes UTF-8 with `errors='replace'`; the tagged chunker runs — `chunk_python` parses the AST, descends big classes, merges small spans (chapter 6); `chunk_markdown` splits at headings outside fences and packs paragraphs (chapter 5); both end at `_make`, where the `<=2000` assert lives.
5. Per chunk: `_add_chunk` runs `analyze()` (tokenize, split identifiers keeping both forms, fold, stoplist — chapter 4), appends to the chunk table, records the term-count length, and folds `Counter(terms)` into `postings[term][chunk_id]`.
6. After the `tqdm` loop: `avgdl` is computed, `save_index` pickles the `Bm25Index` under `data/processed/` and writes the human-readable meta JSON. The returned string is the 'Ingestion complete!' line `Fire` prints.

uv run python -m src search_dataset --dataset_path ... --k 10 --save_directory ...

1. Same dispatch; `cli.search_dataset` loads the dataset through `load_model(Path, RagDataset)` — `pydantic` validates every question here, so a malformed file dies at the boundary with a clean `ValidationError`, translated by the `__main__` error boundary (chapter 10).
2. `load_index` unpickles the `Bm25Index` once — the only expensive load in the whole run.

3. `search_dataset_results` iterates questions under `tqdm`; per question, `search()` calls `bm25_scores`: analyze the query with the SAME analyzer, walk each unique term's postings, accumulate `idf x saturated-normalized-tf` into a dict (chapter 8, and Volume 1 ch. 7 for every constant).
4. `heapq.nlargest` picks the top `k`; each chunk id is materialized as a `MinimalSource` carrying the chunk's stored verbatim path and span — no path manipulation happens at search time, by design.
5. The per-question `MinimalSearchResults` are wrapped in `StudentSearchResults(k=k)`; `save_model` writes pretty JSON under the scoped save directory, creating parents. The returned 'Saved ...' string matches the subject's walkthrough output.

Where each failure would surface

If this breaks...	...you see it here
discovery filters wrong	index meta <code>n_chunks</code> absurdly low/high; explore script's top dirs miss docs/ or vllm/
chunker offset bug	<code>test_spans_reproduce_text</code> fails; or recall sags with nothing crashing
analyzer asymmetry	single-term queries that should hit a known chunk return nothing
path prefix wrong	<code>moulinette</code> validates but recall is exactly 0.000
oversized chunk leaked	<code>moulinette</code> : 'Student data is valid: False'
index not rebuilt after code change	<code>pickle.UnpicklingError</code> , or silently stale chunk table

Notice the pattern: every component failure has exactly one earliest observable symptom, and they are all cheap to check in order. That is not luck — it falls out of the stage contracts. When you can produce this table from memory, you are ready for the “explain your architecture” portion of the defense, and — more usefully — you can debug your own system in minutes instead of evenings.

CHAPTER 15

Passing the Gates: flake8, mypy, Docstrings

make lint is part of the mandatory contract. This chapter makes it boring — the correct emotional state for a linter.

The subject's exact demands: flake8 compliance, mypy with `--warn-return-any` `--warn-unused-ignores` `--ignore-missing-imports` `--disallow-untyped-defs` `--check-untyped-defs`, type hints on parameters and returns, PEP 257 docstrings on functions and classes, and graceful exception handling. Every listing in chapters 1-12 was written against those flags; this chapter explains the recurring patterns so you can keep the property as you modify code.

What each mypy flag actually enforces

Flag	Meaning	Practical consequence
<code>--disallow-untyped-defs</code>	every function must be fully annotated	<code>def f(x): ...</code> is an error; even tests need annotations
<code>--check-untyped-defs</code>	bodies of unannotated defs are still checked	no hiding errors in legacy helpers
<code>--warn-return-any</code>	returning an Any-typed value is flagged	<code>json.load</code> results must be cast/validated before returning
<code>--warn-unused-ignores</code>	a <code>'# type: ignore'</code> that ignores nothing is an error	you cannot sprinkle ignores preventively
<code>--ignore-missing-imports</code>	untyped third-party packages don't error	<code>fire</code> and <code>tqdm</code> import cleanly; their objects type as Any

The five patterns that keep this codebase green

- Annotate at the boundary, trust inside.** Functions taking/returning our own types (Chunk, Bm25Index, models) need no ceremony. The friction concentrates where Any enters: `fire`, `tqdm`, `transformers`. The generator isolates those behind three narrow `'# type: ignore[...]'` comments with specific codes — specific, because `--warn-unused-ignores` turns stale blanket ignores into errors when a library later ships stubs.
- Return precise types from JSON.** `json.load` returns Any; returning it trips `--warn-return-any`. The cure is structural: all JSON passes through `pydantic` (`load_model` returns M), so Any never escapes an I/O helper. This is a case where the type checker pushed the design somewhere better.

3. **Optional is spelled out.** The model cache is `Optional[Tuple[object, object]]` with a `None` check before use; mypy verifies the narrow. Never initialize a global to `None` without `Optional` in the annotation.
4. **Docstrings: one imperative summary line**, blank line, then detail only when the signature doesn't say it all (PEP 257; Google style for Args: sections when arguments need prose, as in `cli.index`). flake8's `pydocstyle` plugins are not mandated — the subject asks for presence and convention, not a specific checker.
5. **Run make lint after every file you touch**, not at the end. The cost curve is brutally convex: fixing types at write time is seconds; retrofitting a finished untyped project is the weekend before the defense.

A worked mypy failure and its fix

The typical journey: the fix for the type error is usually a design fix

```
# BEFORE - two errors under the subject's flags
def top_paths(results):                # untyped def
    data = json.load(open(results))    # unmanaged resource
    return data["paths"]              # returning Any

# AFTER - clean, and better designed
def top_paths(results: Path) -> List[str]:
    """Return the file paths listed in a results JSON."""
    parsed = load_model(results, StudentSearchResults)
    return [
        s.file_path
        for r in parsed.search_results
        for s in r.retrieved_sources
    ]
```

Note — `lint-strict` (mypy `--strict`) additionally demands typed generics everywhere (`dict[str, float]`, not `dict`), no implicit `Optional`, and typed decorators. The listings here are strict-clean or one annotation away — attempt it once your mandatory part is stable; it is explicitly recommended, not required.

CHAPTER 16

Testing What Matters

Tests are not submitted or graded — and you should write them anyway, because they guard exactly the failures that cost recall silently.

The dangerous bugs in this project do not crash: a shifted offset, a desynchronized analyzer, an oversized chunk — each silently costs recall or validity. Unit tests are how you make them loud. Priorities, in order of protection per line of test code:

1 — Chunker invariants (the highest-value tests you will write)

The five chunker tests that catch 90% of silent recall bugs

```
"""tests/test_chunkers.py - invariant tests."""
from pathlib import Path

from src.chunkers import chunk_markdown, chunk_python

SAMPLE_MD = "# Title\n\nPara one.\n\n## Sub\n\nPara two.\n"
SAMPLE_PY = (
    "import os\n\n\n"
    "def f(x: int) -> int:\n"
    '    \\'Docstring.\'"\n'
    "    return x + 1\n"
)

def test_spans_reproduce_text_md() -> None:
    """Slicing the file with a chunk's span yields its text."""
    for c in chunk_markdown("f.md", SAMPLE_MD, 2000):
        assert SAMPLE_MD[c.start:c.end] == c.text

def test_spans_reproduce_text_py() -> None:
    """Same invariant for the AST chunker."""
    for c in chunk_python("f.py", SAMPLE_PY, 2000):
        assert SAMPLE_PY[c.start:c.end] == c.text

def test_size_cap_holds_under_tiny_limit() -> None:
    """No chunk ever exceeds the limit, even a hostile one."""
    long_md = "word " * 5000
    for c in chunk_markdown("f.md", long_md, 100):
        assert c.end - c.start <= 100

def test_heading_starts_chunk() -> None:
    """A chunk boundary falls before the ## heading."""
    chunks = chunk_markdown("f.md", SAMPLE_MD, 20)
    assert any(c.text.startswith("## Sub") for c in chunks)

def test_python_syntax_error_falls_back() -> None:
```

```

"""Broken Python still produces chunks (text fallback)."""
broken = "def f(:\n    pass\n"
assert chunk_python("f.py", broken, 2000)

```

2 — Analyzer contract

Freezing the analyzer's contract

```

def test_identifier_splitting_keeps_both() -> None:
    """Compound identifiers index whole AND parts."""
    terms = analyze("gpu_memory_utilization")
    assert "gpu_memory_utilization" in terms
    assert {"gpu", "memory", "utilization"} <= set(terms)

def test_camel_case_splits() -> None:
    """PagedAttention -> paged, attention (plus the whole)."""
    terms = analyze("PagedAttention")
    assert {"paged", "attention"} <= set(terms)

def test_empty_and_symbols() -> None:
    """Degenerate inputs produce empty term lists, no error."""
    assert analyze("") == []
    assert analyze("!!! ??? **") == []

```

3 — Retrieval sanity and the metric

Retrieval ranking + the exact grader geometry from Volume 1

```

def _tiny_index() -> Bm25Index:
    """Three-chunk corpus with known term distribution."""
    idx = Bm25Index()
    for i, text in enumerate(
        ["paged attention blocks", "scheduler queue",
         "attention is common here attention"]
    ):
        _add_chunk(idx, Chunk(f"f{i}.md", 0, len(text), text))
    idx.avgdl = sum(idx.doc_len) / idx.n_chunks
    return idx

def test_rare_term_outranks_frequency() -> None:
    """One 'paged' beats two 'attention' (IDF beats TF)."""
    sources = search(_tiny_index(), "paged attention", k=2)
    assert sources[0].file_path == "f0.md"

def test_iou_threshold_edges() -> None:
    """The 0.05 bar behaves exactly as the spec's geometry."""
    a = MinimalSource(file_path="f", first_character_index=0,
                      last_character_index=2000)
    near = MinimalSource(file_path="f", first_character_index=1500,
                        last_character_index=1900)
    graze = MinimalSource(file_path="f", first_character_index=1990,
                         last_character_index=2390)
    assert _iou(a, near) >= 0.05
    assert _iou(a, graze) < 0.05

```

Round out with one end-to-end smoke test: build an index over a three-file temporary corpus (`tmp_path` fixture), run `search_dataset` on a two-question `RagDataset`, validate the output parses back as `StudentSearchResults`, and assert every span is ≤ 2000 . That single test exercises discovery, both chunkers, the analyzer, BM25, serialization, and the models — if it passes after a refactor, you probably didn't break the pipeline.

Note — Remember the flags: tests are code too — annotate every test function (`-> None`) or make lint fails on your own test suite. The listings above comply.

CHAPTER 17

Troubleshooting the Build

The twenty failures everyone hits, in the order everyone hits them — with the one-line diagnosis first.

Symptom	Cause	Fix
KeyError: 'qwen3' loading the model	transformers < 4.51 has no qwen3 architecture	pin transformers>=4.51; uv sync
uv sync fails on python version	.python-version older than requires-python	align both (e.g. 3.12); commit both files
ModuleNotFoundError: src	running python src/cli.py directly	always uv run python -m src <cmd> from the repo root
Fire prints help instead of running	method name typo'd or private (leading _)	public method on RagCli; check --help output
flake8 reports thousands of errors	linting the vLLM corpus and .venv	.flake8 exclude = .venv,data (chapter 1)
mypy: 'Function is missing a type annotation'	--disallow-untyped-defs applies to every def, tests too	annotate; return None means '-> None'
Recall is 0.000 but search 'looks right'	file_path prefix mismatch vs data/raw/vllm-0.10.1/...	store as_posix() relative paths from discovery on
Recall lower than expected on code only	no identifier splitting, or analyzer asymmetry	one analyze() imported by both sides; test ch. 14
'Student data is valid: False' from moulinette	a source wider than 2000 chars, or schema drift	the _make assert; validate output before writing
Indexing exceeds 5 minutes	indexing huge/binary files, or O(n^2) string concat	size cap + extension filter; build lists, join once
Answers start with pages of reasoning	thinking mode active	enable_thinking=False in apply_chat_template + _clean
Answer generation impossibly slow	reloading the model per question	module-level cache (_load_model), one load per process
Offsets look shifted by 1-2 chars on some files	\r\n handling or a re-decoded file	single read_text; splitlines(keepends=True) everywhere
Same output file overwritten between datasets	unscoped --save_directory	scope by dataset as the walkthrough shows
pickle.UnpicklingError on load_index	index built with different code version	re-run index after changing Chunk/Bm25Index shape

Symptom	Cause	Fix
Works locally, dies at defense on 'No space left'	HF cache + torch on a small partition	subject's warning: run from a disk with several GB free
tqdm bars garble CLI output piped to files	progress written to stdout	tqdm writes to stderr by default - don't redirect 2>
Fire swallows --k=banana as a string	Fire coerces only when unambiguous	int() casts at method entry (chapter 9) + boundary
ValidationError: answered question parsed as unanswered	Union order in RagDataset	AnsweredQuestion first in the Union (chapter 2)
Evaluate disagrees with moulinette	IoU/macro definition drift in your evaluation.py	re-derive from Volume 1 ch. 11; test the edge cases

Debugging discipline when a symptom is not in the table: (1) reproduce on the smallest input that fails — one file, one question; (2) bisect the pipeline at its data contracts — is the chunk right? the terms? the score? the JSON? — the stage boundaries from Volume 1 exist precisely so failures localize; (3) only then read code. Most RAG bugs are data bugs wearing a code costume.

CHAPTER 18

Build Checklist and Bridge to Volume 3

Everything Volume 2 promised, verifiable in one sitting.

The mandatory-part checklist

1. uv sync from a clean clone succeeds; uv.lock committed; .python-version present.
2. make install / run / debug / clean / lint all exit 0; lint runs the subject's exact mypy flags.
3. All eight pydantic models match the subject; load/save helpers validate at every I/O boundary.
4. index builds under 5 minutes; data/processed/ contains the index plus readable meta.
5. Both chunkers hold the ≤ 2000 invariant under hostile inputs (tests pass); offsets slice-exact against raw files.
6. search returns top-k MinimalSources with verbatim data/raw/vllm-0.10.1/... paths.
7. search_dataset writes StudentSearchResults JSON that the moulinette validates (run it!).
8. answer / answer_dataset produce clean grounded answers; thinking stripped; empty-retrieval guard works.
9. evaluate reproduces the metric; numbers within noise of the moulinette on public datasets.
10. Every edge case in chapter 10's table exits cleanly — verified against the real CLI, exit codes checked.
11. The four-command walkthrough (chapter 12) runs verbatim from a fresh clone next to the datasets.

What Volume 3 adds

At this point you have a complete, compliant, robust system — which is necessary and not sufficient: nothing yet guarantees 80%/50% recall@5. Volume 3 is the measurement volume: moulinette internals and failure triage, a tuning harness for chunk size and BM25 parameters, per-corpus index strategies, the performance audit, all five bonuses implemented on top of the code you just built (each one a bounded diff), the README the subject demands section by section, and the defense itself — including the recode scenarios evaluators actually use. The system is built; now make it score.

Submission hygiene

Only the work inside the repository is evaluated. A last audit before you consider the build done:

- git ls-files should show: src/ modules, tests/, Makefile, README.md, pyproject.toml, uv.lock, .gitignore, .python-version, .flake8 — and no data, no index, no output JSON, no model weights, no .venv, no moulinette binary.
- Clone your own repo into a fresh directory and run the chapter-12 walkthrough from scratch, timing the index step. This clone-and-run rehearsal is the exact experience your evaluator has; every friction you feel, they feel with less patience.
- Check file names character by character: Makefile (not makefile), README.md at the root, src runnable as -m src. The subject closes by telling you to double-check names — it is not a rhetorical flourish, renames are a classic zero.
- Tag or note the commit you consider final. Defenses have a way of starting from 'wait, which version is on the server?' — remove the question.

Note — A useful habit for the whole week: commit at every green make lint with a message naming the chapter you just completed ('chunkers: markdown strategy + tests'). The history becomes your build log, and reverting a bad experiment is one command instead of an archaeology session.